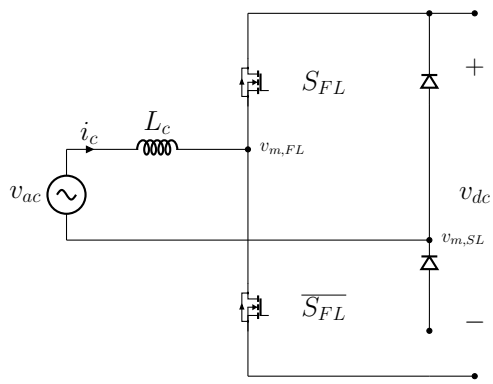
Single-Phase Totem-Pole PFC

Control in the abc frame

Kristian Skorpen

1 Introduction



In this course we cover the **single-phase totem-pole PFC** and how to control its input current (i_c) and active power drawn from the grid. The totem-pole PFC is a rectifier: power flows from the AC grid into the DC bus, and the control target is a sinusoidal input current in phase with the grid voltage (unity power factor).

The course is built around control in the **stationary frame** (abc), with derivations and runnable numerical examples in Python.

1.1 Learning outcomes

After completing this course you will be able to:

1. Explain the topology of a single-phase totem-pole PFC and its fast-leg / slow-leg operation.
2. Derive the duty-cycle reference for a PWM modulator.
3. Implement a PI+feedforward current controller in the abc frame.
4. Implement an active power controller and a DC-link voltage controller, and decide which to use for a given application.

1.2 Prerequisites

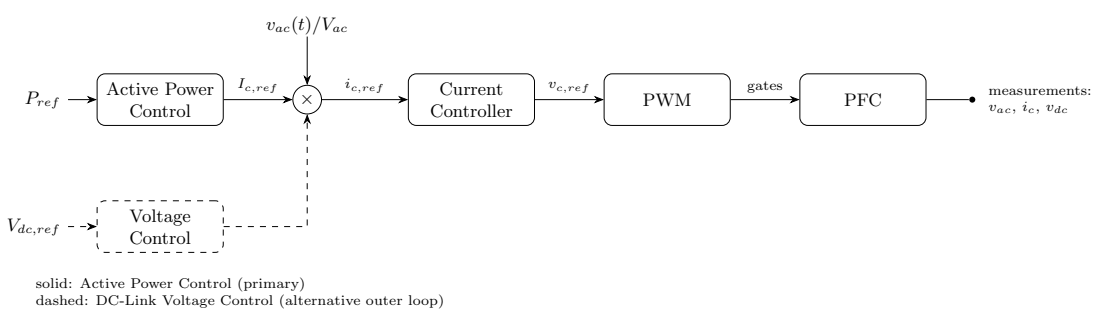
To take the course, I recommend a basic understanding of electronics, math, and control theory. I expect that you understand, at a surface level, how inductors and PI controllers work. However, you do not need to understand how a totem-pole PFC works or how it is controlled. These topics are covered in the course.

1.3 Notation (used throughout)

Symbol	Meaning	Units
v_{ac}, i_c	Grid-side voltage and converter output (inductor) current, time-domain	V, A
V_{ac}, I_c	**RMS** values of the same quantities (capital letters for RMS)	V, A
v_c	Converter voltage (the PFC's controlled output)	V
v_{dc}	DC-link voltage	V
L_c	Output inductance	H
$\omega = 2\pi f_{ac}$	Grid angular frequency	rad/s
θ	Grid voltage angle	rad
$\hat{\theta}$	PLL estimate of θ	rad
v_α, v_β	Clarke-transformed voltages (stationary)	V
v_d, v_q, i_d, i_q	Park-transformed voltages and currents (rotating)	V, A
D_{FL}, S_{SL}	Fast-leg duty cycle, slow-leg gate signal	-, {0,1}
S_{FL}	Fast-leg gate signal	{0,1}
P, Q	Active and reactive power (average)	W, var

Lower-case letters denote instantaneous time-domain quantities; upper-case letters denote RMS values. Subscript *ref* means "commanded"; hat ($\hat{\cdot}$) means "estimated".

1.3.1 Control Structure in the abc frame



Contents

1 Introduction **1**
- 1.1 Learning outcomes . . . 1
- 1.2 Prerequisites . . . 1
- 1.3 Notation (used throughout) . . . 1
  - 1.3.1 Control Structure in the abc frame . . . 2

2 The Totem-Pole PFC **4**
- 2.1 Topology . . . 4
- 2.2 Simplified model of the PFC . . . 5
- 2.3 Control Structure . . . 6

3 PWM Strategy **6**
- 3.1 Gate Signal Derivation . . . 7
- 3.2 Common setup (run this block once, then every numerical example reuses it) . . . 8
  - 3.2.1 Numerical Example . . . 8

4 Control in abc frame **9**
- 4.1 Current Control in abc Frame . . . 10
  - 4.1.1 Derivation of the Reference Voltage . . . 10
  - 4.1.2 Implementation of PI Controller . . . 10
  - 4.1.3 Numerical Example . . . 11
  - 4.1.4 Pros and Cons . . . 13
- 4.2 Power Calculation Methods . . . 14
  - 4.2.1 RMS-based Definitions . . . 14
  - 4.2.2 Alternative calculation Methods . . . 15
  - 4.2.3 Numerical Example (For deeper understanding) . . . 16
  - 4.2.4 Pros and Cons . . . 17
- 4.3 Pure Active Power Control . . . 18
  - 4.3.1 Control Method . . . 18
  - 4.3.2 Implementation of PI Controller . . . 18
  - 4.3.3 Numerical Example . . . 19
  - 4.3.4 Pros and Cons . . . 22
- 4.4 DC-Link Voltage Control . . . 22
  - 4.4.1 Control Method . . . 23
  - 4.4.2 Pros and Cons . . . 23

5 Comparison of Control Methods **23**

2 The Totem-Pole PFC

2.1 Topology

A simplified model of a totem-pole PFC is shown in Figure 1. On the left is a DC link, and on the right are an AC voltage source and an inductor. In between, we have two switching legs. We use *fast leg* (FL) and *slow leg* (SL) to refer to the left and right switching legs, respectively. S_{FL} and S_{SL} are the gate signals for the upper switches. Only one switch can be on at a time in each leg, as indicated by the overline.

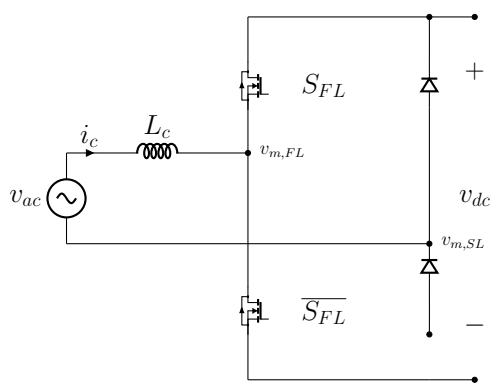


Figure 1: Totem-pole PFC

The switching legs are called fast and slow because they switch at different frequencies. The slow leg switches at twice the AC voltage source's fundamental frequency. While the fast leg switches at a much higher frequency. The fast-leg / slow-leg switching strategy is well suited for a PFC rectifier and is a natural choice for a bidirectional totem-pole PFC. If you design your own PFC, you will pick a switching strategy to fit your goals (efficiency, EMI, device count). The specific PWM choice does not change the current- or power-control design that follows, this course is PWM-agnostic from the next chapter onward. All chapters, except the PWM chapter, are relevant to all types of PWM strategies. However, I will use the names "fast leg" and "slow leg" to distinguish between the two legs during switching.

Assumptions used throughout the rest of the course:

- Switches are ideal (zero on-resistance, instantaneous turn-on/off, no dead-time).
- The DC-link voltage v_{dc} is constant and large enough that over-modulation does not occur (see PWM chapter for the explicit bound).
- The output inductor L_c is linear (no core saturation, no equivalent series resistance).
- The grid is modelled as a stiff sinusoidal voltage source $v_{ac}(t) = \sqrt{2}\,V_{ac}\sin(\omega t)$.
- No measurement or communication delay, every signal is available instantaneously.
- Sampling time Δt is much shorter than the slowest controller time constant; discrete-time effects are discussed in the appendix.

These are idealisations. The *When to use each method* chapter and the appendix revisit them.

2.2 Simplified model of the PFC

We can further simplify the model. If we look at $v_{m,SL}$ in the slow leg, when the upper switch is open ($S_{SL} = 0$), $v_{m,SL} = 0$ and when it is closed ($S_{SL} = 1$), $v_{m,SL} = v_{dc}$. v_{dc} is reference to ground.

$$v_{m,SL} = \begin{cases} 0, & S_{SL} = 0 \\ v_{dc}, & S_{SL} = 1 \end{cases}$$

We can simplify this expression as:

$$v_{m,SL} = S_{SL}\, v_{dc}$$

And we can do the same for the fast leg:

$$v_{m,FL} = \begin{cases} 0, & S_{FL} = 0 \\ v_{dc}, & S_{FL} = 1 \end{cases}$$

$$v_{m,FL} = S_{FL}\, v_{dc}$$

Because the fast leg and the slow leg can be modelled as voltages, we can replace them with controlled voltage sources, as shown in Figure 2. If we set the voltage reference (Ground) between the ac voltage (v_{ac}) and the slow leg ($v_{m,SL}$), as shown in Figure 2. We would get Figure 3, where v_c is the converter's output voltage.

$$v_c = v_{m,FL} - v_{m,SL}$$

Ground is not shown explicitly: it is a reference and could be placed anywhere.

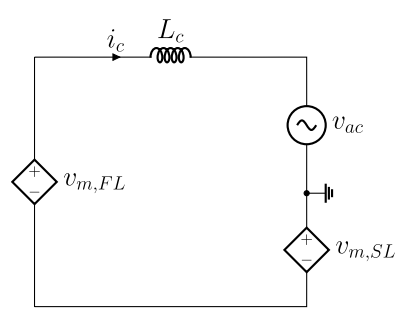


Figure 2: Fast-leg and slow-leg replaced by controlled voltage sources. The output voltage is v_c = v_m,FL - v_m,SL.

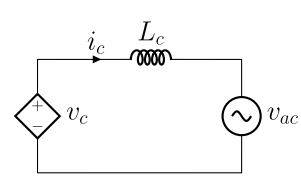
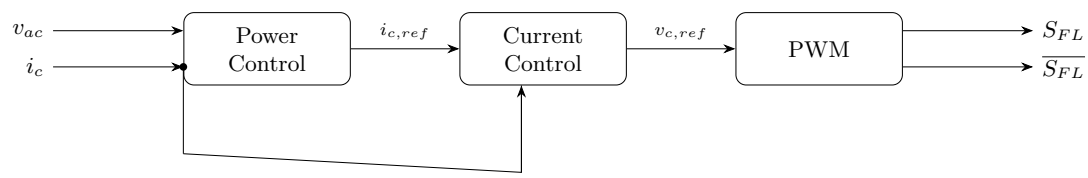


Figure 3: Equivalent circuit used for the rest of the course the PFC as a controlled voltage source v_c in series with inductor L_c, feeding the grid voltage v_ac.

From Figure 3 onwards, *"the PFC"* means this equivalent circuit: a controlled voltage source v_c in series with the output inductor L_c, feeding the stiff AC voltage v_{ac}. Our job as control designer is to pick $v_c(t)$ so that the inductor current $i_c(t)$ follows a reference. The fast leg and the slow leg are the physical actuators that realise v_c.

2.3 Control Structure

On a high level the control of a totem-pole PFC looks like this:



The controller is split into three blocks: **Power control**, **Current control**, and **PWM**. Each block operates on a different timescale. PWM is an analog signal, meaning a continuous signal, while the current controller is sampled in microseconds and the power controller samples in milliseconds. The sampling times depend heavily on application, but the sampling time of the current controller is significantly lower than the sampling time for the power controller. Each block can therefore be designed on the assumption that the block below it has already settled, which is a standard and extremely effective simplification.

First, we have the Power Control, which calculates the desired current reference ($i_{c,ref}$) based on the measurements and the desired Power. It is desired that the reference current ($i_{c,ref}$) changes slowly in order not to introduce harmonics. Therefore, the Power Controller can be sampled less often than the other block.

Then we have the Current Control, which calculates the reference voltage ($v_{c,ref}$) for the PWM to achieve the desired current (i_c).

This controller does not control the DC-link voltage (v_{dc}). Whether the PFC should control the DC-link voltage (v_{dc}) depends on the system around the PFC and the primary goals. We do not go into detail on that here; if you want to control the dc-link voltage (v_{dc}), you can replace the Power Control with a Voltage Control. The input of the Voltage Controller would be the dc-link voltage (v_{dc}) and the desired dc-link voltage, while the output would be the reference current ($i_{c,ref}$), as for the Power Controller. Whether you have a Power Controller or a Voltage Controller does not affect the Current Controller or the PWM. The Voltage Controller works well for unity power factor operations, and where computational simplicity is important.

3 PWM Strategy

The PWM receives a reference voltage ($v_{c,ref}$) and generates gate signals that drive the PFC. The reference voltage ($v_{c,ref}$) is given by the following equation as described above:

$$v_{c,ref} = v_{m,FL} - v_{m,SL}$$

3.1 Gate Signal Derivation

Remember that $v_{m,FL}$ and $v_{m,SL}$ are given by:

$$v_{m,FL} = S_{FL}\, v_{dc}, \qquad v_{m,SL} = S_{SL}\, v_{dc}.$$

Applying the fast-leg gate signal directly is computationally prohibitive: at a 100 kHz switching frequency the outer controller would need to resolve the gate state hundreds of thousands of times per second. Instead we use an **average model** (also called fundamental-frequency averaging): we compute a *duty cycle* $D_{FL} \in [0, 1]$, the average of S_{FL} over one switching period, and let an analog comparator turn that into the actual gate signal at the carrier frequency. The slow leg switches only twice per grid period, so its gate signal S_{SL} is kept as-is.

The average-model replacement is

$$v_{m,FL} = D_{FL}\, v_{dc}.$$

Substituting into $v_{c,ref} = v_{m,FL} - v_{m,SL}$:

$$v_{c,ref} = D_{FL}\, v_{dc} - S_{SL}\, v_{dc}.$$

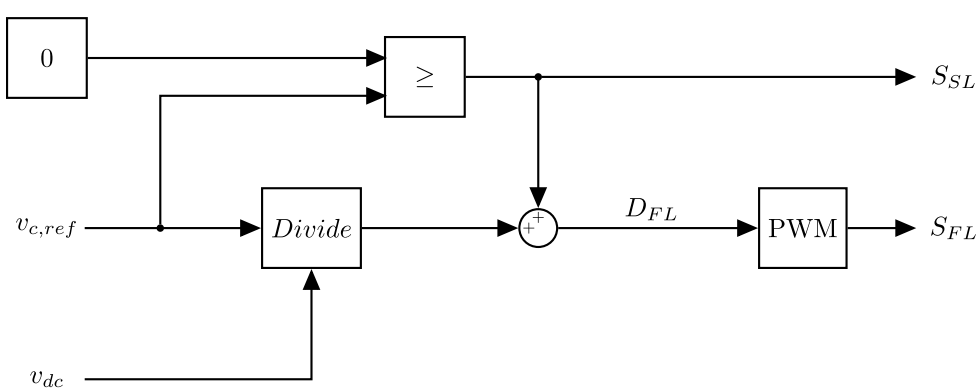
The unknowns are D_{FL} (continuous, $[0, 1]$) and S_{SL} (binary, $\{0, 1\}$). Two cases:

- **If** $v_{c,ref} \leq 0$, choose $S_{SL} = 0$ so $v_{m,SL} = 0$. Then $v_{c,ref} = D_{FL}\, v_{dc}$, but that would force D_{FL} negative when $v_{c,ref} < 0$, impossible. So instead we pick $S_{SL} = 0$ when $v_{c,ref} > 0$ and $S_{SL} = 1$ when $v_{c,ref} \leq 0$, which *shifts* the fast-leg operating range into a feasible interval.
- Solving the two-case system gives the compact form:

$$S_{SL} = \begin{cases} 0 & \text{if } v_{c,ref} > 0 \\ 1 & \text{if } v_{c,ref} \leq 0 \end{cases}, \qquad D_{FL} = S_{SL} + \frac{v_{c,ref}}{v_{dc}}.$$

The equation is solvable only when $|v_{c,ref}| \leq v_{dc}$. Outside that range the PFC is *over-modulated*: the converter simply cannot produce the commanded voltage and the duty-cycle would have to exceed $[0, 1]$.

The overall block diagram of the PWM would look like this:



3.2 Common setup (run this block once, then every numerical example reuses it)

All numerical examples below share imports, plotting helpers, and the baseline grid parameters. Run the two cells in this section **once at the start** of your session. Every later *Numerical Example* block will then work without re-declaring anything.

[2]:
```
import numpy as np
import importlib
import single_phase_inverter_functions as inv_func

_ = importlib.reload(inv_func)
```

Common variables for all numerical examples:

[3]:
```
Vac = 230       # RMS value of AC voltage
fac = 50        # Frequency of AC voltage
Vdc = 400       # Constant DC link voltage
Lc = 1e-3       # Inductance

# Defining time
t_end = 1/fac
N = 1000 # Number of samples
t = np.linspace(0, t_end, N)
```

3.2.1 Numerical Example

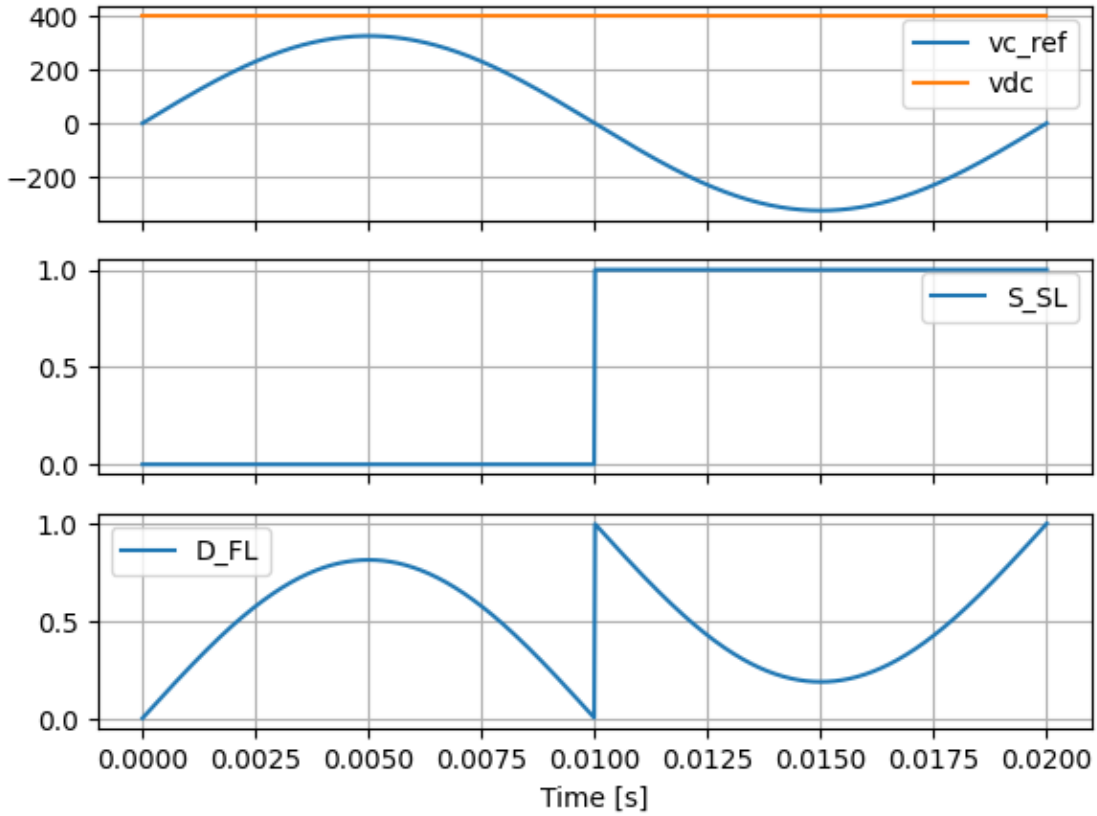
Defining Parameters

[4]:
```
# Making a sine wave for vc_ref (Close to equivalent to Vac)
vc_ref= np.sqrt(2) * Vac * np.sin(2 * np.pi * fac * t)
```

Calculating PWM

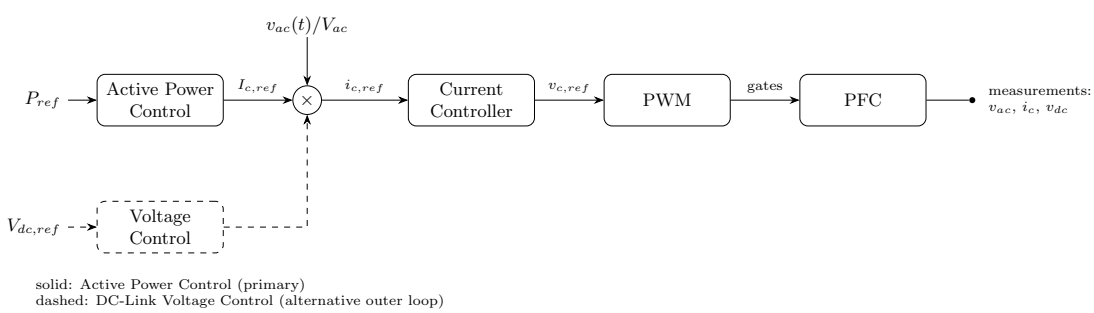
[5]:
```
# S_SL is 1 when v_c,ref <= 0 and 0 when v_c,ref > 0 (equation in the␣
↪derivation above).
# Note: (vc_ref < 0) gives the correct boundary up to the equal-zero case,␣
↪which is
# numerically measure-zero for a sinusoidal vc_ref.
S_SL = (vc_ref <= 0).astype(int)

D_FL = S_SL + vc_ref / Vdc

inv_func.plot_time_signals(
    t,
    [
        [[vc_ref, "vc_ref"], [Vdc, "vdc"]],
        [[S_SL, "S_SL"]],
        [[D_FL, "D_FL"]],
```

```
    ]
)
```



4 Control in abc frame

The figure below gives an overview of how a single-phase bidirectional totem-pole PFC can be controlled in the abc frame.



4.1 Current Control in abc Frame

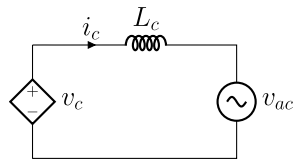


Figure 3: Equivalent circuit used for the rest of the course the PFC as a controlled voltage source v_c in series with inductor L_c, feeding the grid voltage v_ac.

The current we want to control is the output current, which is equivalent to the inductor current (i_c). If we have a filter on the output, it becomes more complicated, but the main principles are the same. In this section, we will derive a current controller with PI and feedforward.

4.1.1 Derivation of the Reference Voltage

The inductor current depends on the voltage across the inductor. Writing KVL around the loop in Figure 3:

$$v_c \;-\; v_{L_c} \;-\; v_{ac} \;=\; 0 \qquad \Longrightarrow \qquad v_c \;=\; v_{L_c} \;+\; v_{ac}.$$

The inductor defining equation is

$$v_{L_c} \;=\; L_c \,\frac{di_c}{dt}.$$

Combining the two:

$$v_c \;=\; L_c \,\frac{di_c}{dt} \;+\; v_{ac}.$$

This is the *plant* equation: it tells us what v_c we would need if i_c followed its reference perfectly. Setting $v_c = v_{c,ref}$ and using a forward-Euler approximation $di_c/dt \approx (i_{c,ref}[k] - i_c[k-1])/\Delta t$:

$$v_{c,ref}[k] \;=\; \frac{L_c}{\Delta t}\,(i_{c,ref}[k] - i_c[k-1]) \;+\; v_{ac}[k].$$

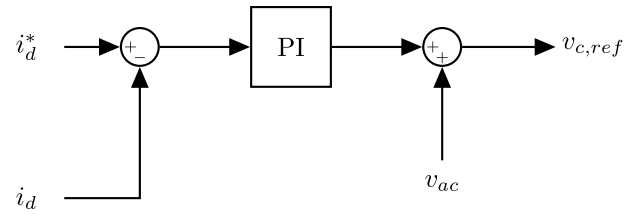
This is the **feedforward** reference: if the plant model were perfect and we had no noise we would be done. The next section adds a PI controller to cover for model mismatch and disturbances.

4.1.2 Implementation of PI Controller

The feedforward equation above assumes a perfectly known inductor, noise-free measurements, and zero delay. None of those hold in practice, so we add a PI controller on the tracking error to handle the residual:

$$v_{c,ref} \;=\; \underbrace{\frac{L_c}{\Delta t}\,(i_{c,ref} - i_c)}_{\text{discrete feedforward}} \;+\; \underbrace{v_{ac}}_{\text{grid feedforward}} \;+\; \underbrace{\mathrm{PI}(i_{c,ref} - i_c)}_{\text{correction}}.$$

In many industrial implementations the discrete-feedforward term $L_c\, di_c/dt$ is *dropped* because the numerical derivative of a noisy current measurement amplifies that noise.

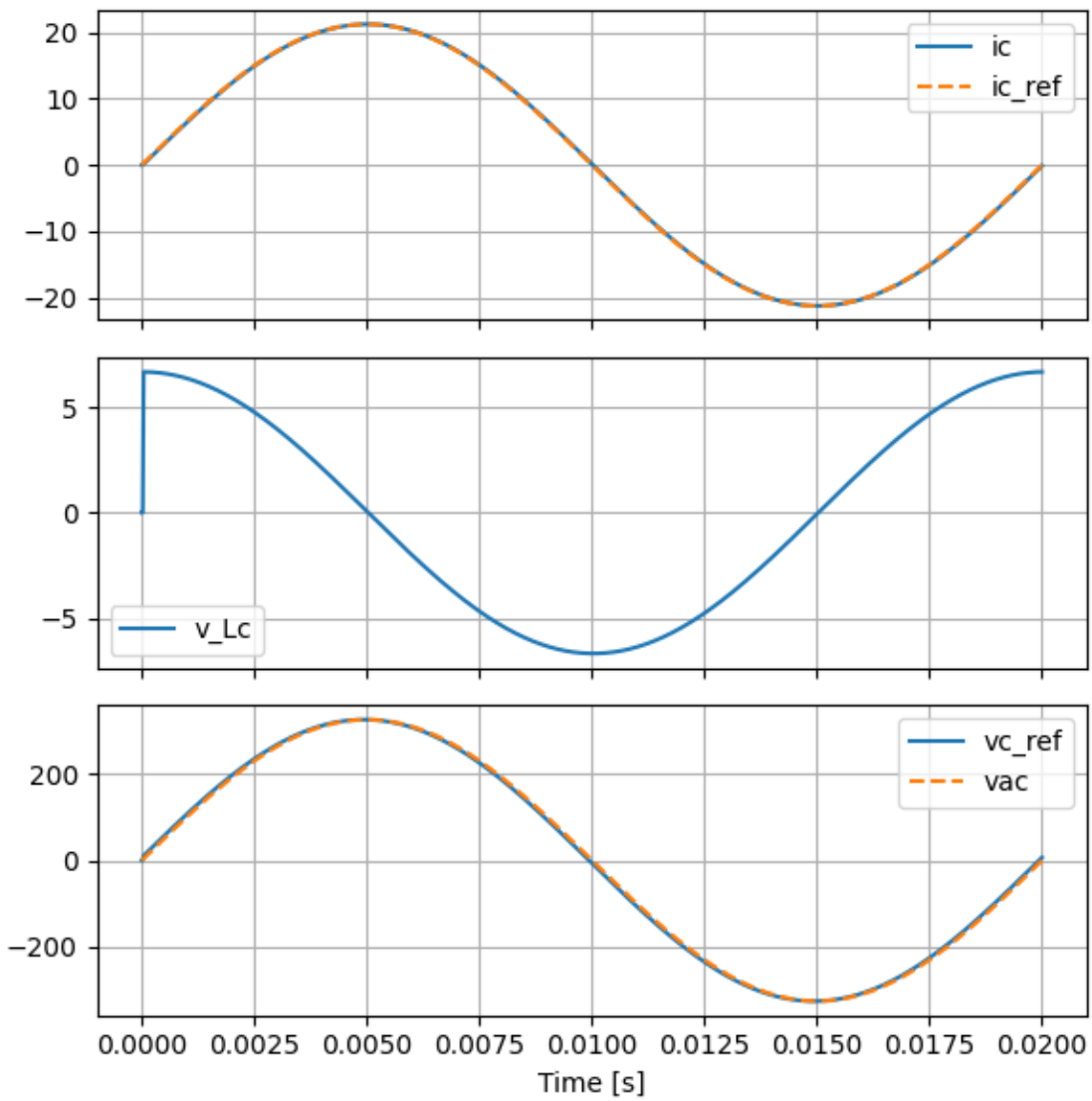


4.1.3 Numerical Example

(This example reuses the imports and grid parameters from the ***Common setup*** *section at the top. Run those cells first.)*

[6]:
```
# Making a sine wave for vac
vac= np.sqrt(2) * Vac * np.sin(2 * np.pi * fac * t)

# Making a reference current
Iac = 15 # RMS value of AC current
ic_ref = np.sqrt(2) * Iac * np.sin(2 * np.pi * fac * t)
```

[7]:
```
# --- Implementation (1 of 3): pure feedforward, no PI ---
# Plant + model are identical in this ideal example, so perfect tracking.
# Δt here is t[1]-t[0]; ensure Δt << 1 / switching frequency (Nyquist).
N_pwm = len(t)
v_Lc = np.zeros(N_pwm)
ic   = np.zeros(N_pwm)
vc_ref = np.zeros(N_pwm)
ic[0] = 0.0  # seed: assume zero initial current

for i in range(1, N_pwm):
    dt = t[i] - t[i-1]
    v_Lc[i] = Lc / dt * (ic_ref[i-1] - ic[i-1])
    ic[i]   = ic[i-1] + v_Lc[i] * dt / Lc

vc_ref = v_Lc + vac

inv_func.plot_time_signals(
    t,
    [
        [[ic, "ic"], [ic_ref, "ic_ref", "--"]],
        [[v_Lc, "v_Lc"]],
        [[vc_ref, "vc_ref"], [vac, "vac", "--"]],
    ],
    height_per_plot=2
)
```



[ ]:
```
# --- Implementation (2 of 3): pure PI, no feedforward ---
# No model term, only PI on the current error. Needs higher gains to track.
Kp_pi = 5.0
Ki_pi = 200.0

ic_pi    = np.zeros(N_pwm)
vc_ref_pi = np.zeros(N_pwm)
err_cum  = 0.0

for i in range(1, N_pwm):
    dt = t[i] - t[i-1]
    err = ic_ref[i-1] - ic_pi[i-1]
    err_cum += err * dt
```

```
    v_cmd = Kp_pi * err + Ki_pi * err_cum       # PI output drives v_L
    ic_pi[i] = ic_pi[i-1] + v_cmd * dt / Lc
    vc_ref_pi[i] = v_cmd + vac[i]

inv_func.plot_time_signals(
    t,
    [
        [[ic_pi, "ic (PI only)"], [ic_ref, "ic_ref", "--"]],
        [[vc_ref_pi, "vc_ref"], [vac, "vac", "--"]],
    ],
    height_per_plot=2
)
```

[ ]:
```
# --- Implementation (3 of 3): feedforward + PI (production pattern) ---
# Feedforward gives the bulk of the command; PI absorbs model error.
Kp_pi = 5.0
Ki_pi = 200.0

ic_ff_pi    = np.zeros(N_pwm)
vc_ref_ff_pi = np.zeros(N_pwm)
err_cum     = 0.0

for i in range(1, N_pwm):
    dt = t[i] - t[i-1]
    err = ic_ref[i-1] - ic_ff_pi[i-1]
    err_cum += err * dt
    v_ff = Lc / dt * (ic_ref[i] - ic_ff_pi[i-1])
    v_pi = Kp_pi * err + Ki_pi * err_cum
    v_cmd = v_ff + v_pi
    ic_ff_pi[i] = ic_ff_pi[i-1] + v_cmd * dt / Lc
    vc_ref_ff_pi[i] = v_cmd + vac[i]

inv_func.plot_time_signals(
    t,
    [
        [[ic_ff_pi, "ic (FF+PI)"], [ic_ref, "ic_ref", "--"]],
        [[vc_ref_ff_pi, "vc_ref"], [vac, "vac", "--"]],
    ],
    height_per_plot=2
)
```

4.1.4 Pros and Cons

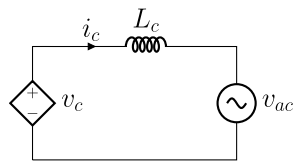
✓ **Pros**

- Simple implementation
- No angle estimation (PLL)
- Low computational cost

✗ **Cons**

- Control variables are sinusoidal
  - Harder to control
  - Harder to analyze response
- Inductor voltage is not a part of feedforward

4.2 Power Calculation Methods



4.2.1 RMS-based Definitions

We want to measure and control the power delivered to the load/grid. The apparent power is given by:

$$S = V_{ac,RMS}\, I_{c,RMS}$$

The real value of S is the active power:

$$P = V_{ac,RMS}\, I_{c,RMS} \cos(\varphi)$$

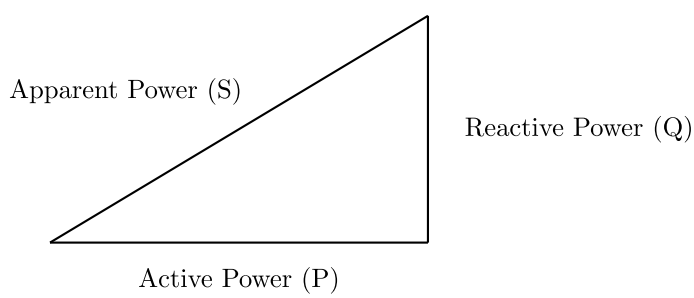
And the imaginary value is the reactive power:

$$Q = V_{ac,RMS}\, I_{c,RMS} \sin(\varphi)$$

The phase angle (φ) is the angle between the voltage and current.

The power triangle is shown below, and from it we know that:

$$S = \sqrt{P^2 + Q^2}$$



For currents and voltages, capital letters denote RMS values, while lowercase letters denote time-domain values. For example, V_{ac} is the RMS value of the AC voltage, while v_{ac} is the time-domain value of the AC voltage. We use this convention from now on. However, this is only for currents and voltages. For example, P is given in the time domain.

4.2.2 Alternative calculation Methods

The problem with the calculation of P and Q shown above is that we need to measure the voltage angle (θ) to calculate φ. In a single-phase system, it is hard to measure the voltage angle. Therefore, The alternative way to calculate P and Q.

Simple method The instantaneous power is

$$p(t) \;=\; v_{ac}(t)\, i_c(t).$$

Both signals are sinusoidal, so $p(t)$ contains a DC component (the average power P) plus a ripple at twice the grid frequency. We want the average:

$$P \;=\; \langle v_{ac}\, i_c \rangle \;=\; \mathrm{LPF}\big(v_{ac}(t)\, i_c(t)\big).$$

This is the standard industry method. It needs only v_{ac} and i_c plus a low-pass filter to extract the average. From now on lowercase p means the instantaneous product and uppercase P means its time-average.

For the reactive power Q we can use the power triangle:

$$Q \;=\; \sqrt{S^2 - P^2}, \qquad S \;=\; V_{ac,RMS}\, I_{c,RMS}.$$

This gives the magnitude only; the sign is lost. We don't know whether it is positive or negative. However, you can find the sign of Q by looking at the angle between the voltage and the current. If the angle is positive (inductive), Q is positive, and if the angle is negative (capacitive), Q is negative.

This way of calculating Q is not used in industry, but it is nice to know.

Advanced method (For Deeper Understanding) This method requires information about the voltage angle (θ). It gives good intuition for how P and Q behave, so we sketch it briefly; it is not used in our controller. This method is inspired by the synchronous-machine power-flow equations. It is not the one we use in the controller, but understanding it is valuable.

Here are the formulas for P and Q:

$$P = \frac{V_c V_{ac}}{\omega L_c} \sin\delta$$

$$Q = \frac{V_{ac}}{\omega L_c}(V_c \cos\delta - V_{ac})$$

The power angle (δ) is the angle between v_c and v_{ac} and is given by:

$$\delta = \theta_{v_c} - \theta_{v_{ac}}$$

If δ is close to 0 and V_c is close to V_{ac}. We have the following properties:

$$\delta \approx 0 \text{ and } V_c \approx V_{ac}$$

A change in δ will result in a change in P and a small change in Q.

$$\text{If } \Delta\delta > 0 \text{ then: } \begin{cases} \Delta P > 0 \\ \Delta Q \approx 0 \end{cases}$$

While a change in V_c will result in a small change in P, a change in Q will result.

$$\text{If } \Delta V_c > 0 \text{ then: } \begin{cases} \Delta P \approx 0 \\ \Delta Q > 0 \end{cases}$$

For a totem-pole PFC, δ is small and V_c is close to V_{ac}. This means that we can control P by controlling δ and control Q by controlling the magnitude of v_c.

We will not use these relationships in the controller, but they provide a good understanding of how P and Q are controlled, since that is essentially how the controller works. However, grid-forming inverters use these equations to control P and Q.

4.2.3 Numerical Example (For deeper understanding)

*(This example reuses the imports and grid parameters from the **Common setup** section at the top. Run those cells first.)*

Here is a numerical example of how P and Q depend on the angle between v_c and v_{ac} and the magnitude of v_c. As explained in the advanced method above. And we will use the simple method to calculate P and Q.

[8]:
```
# Defining time
periods = 2
t_end_long = periods/fac
N_long = 10000 # Number of samples
t_long = np.linspace(0, t_end_long, N_long)

N_period = N / periods

# Making a sine wave for vac
vac= np.sqrt(2) * Vac * np.sin(2 * np.pi * fac * t_long)
```

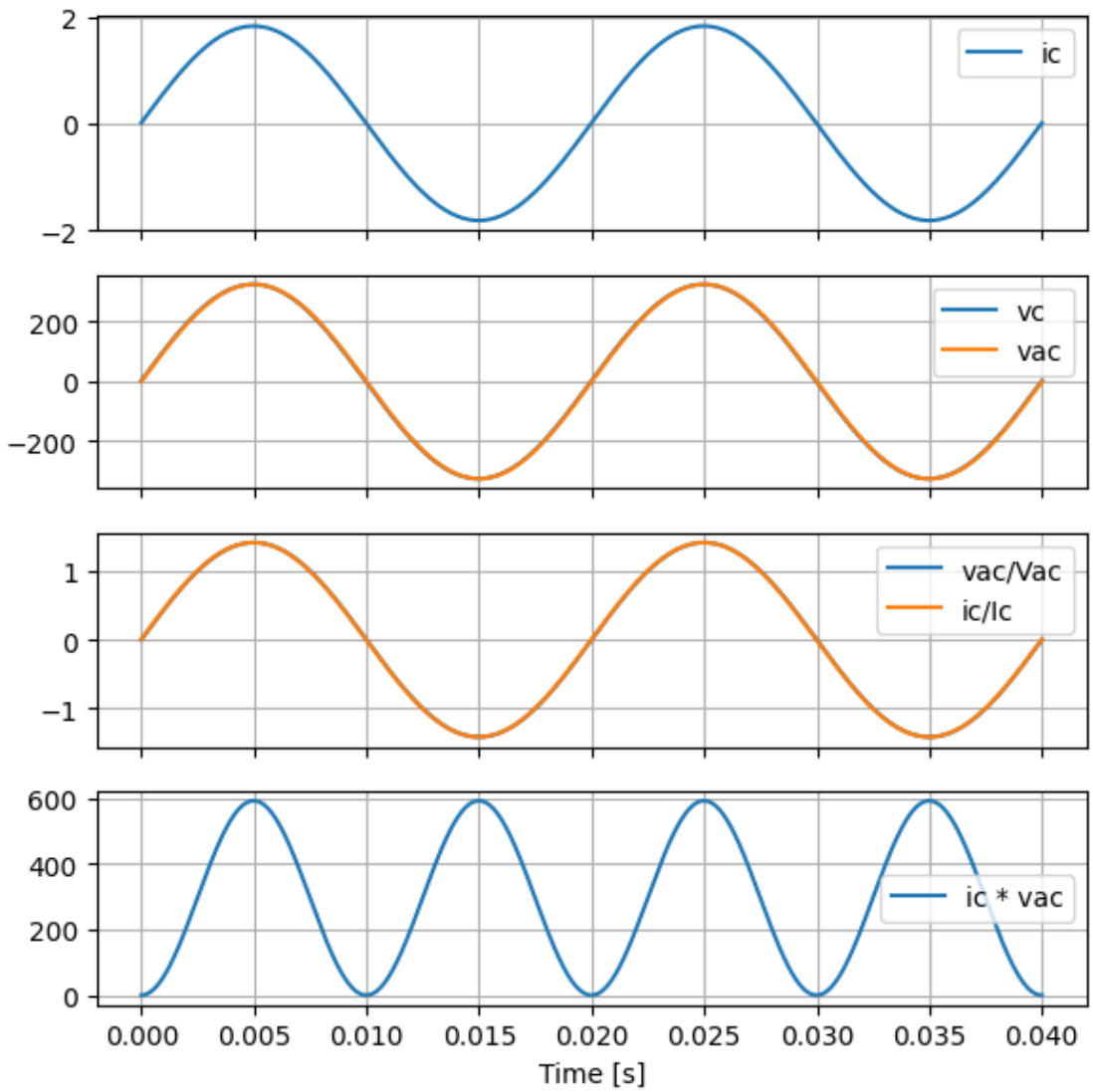
Increase Active Power -> Increase delta_deg

Increase Reactive Power -> Increase mag_scale

[9]:
```
Lc_2 = 5e-3      # Inductance. It is deliberately oversized for illustration
delta_deg = 5
mag_scale = 1.0
```

```
inv_func.calc_and_plot_Power(t_long, vac, delta_deg, N_period, mag_scale, Lc_2)␣
↪# This function can be found in the helper file
```

```
S = 0.30 kVA
P_avg = 0.30 kW
Q_est = -0.00 kVAr
pf = 1.00
```



4.2.4 Pros and Cons

✓ **Pros**

- Simple implementation
- No angle estimation (PLL)

- Low computational cost

✗ **Cons**

- Contains 2ω ripple
- Requires low-pass filtering (adds delay)
- Reactive power estimation requires additional processing

4.3 Pure Active Power Control

Calculating the phase angle (φ) is not easily done for a totem-pole PFC. A simple way to control active power with a reactive-power reference of zero is as follows. This is the same control method used by a **power-factor-correction rectifier (PFC)**: the only target is unity power factor, i.e. $Q = 0$.

If the reactive power is zero, then the voltage and current are in phase ($\varphi = 0$). The idea of this control method is that we assume that the AC voltage (v_{ac}) is sinusoidal, and that the inductor current (i_c) is controlled to have exactly the same waveform as the AC voltage (v_{ac}). The desired inductor current is then obtained by scaling the AC voltage to get the desired power:

$$i_{c,ref} = \frac{I_{c,ref}}{V_{ac}} v_{ac}$$

Where $I_{c,ref}$ is the RMS value of the reference current.

4.3.1 Control Method

The active power is found as described in the simple method above:

$$P = v_{ac}\ i_c$$

The power has a ripple at twice the grid frequency. This is not a problem for the PI controller provided its closed-loop bandwidth is well below 2ω. Concretely, at a 50 Hz grid the ripple sits at 100 Hz; a low-pass corner below ~20 Hz rejects it by ≥14 dB. In practice use an explicit low-pass on the measured P before the PI, rather than relying on the PI to do the averaging.

When designing a PI controller, we want a linear relationship between the input and the output. The average power should be constant in a steady state, and the power is used to control the sinusoidal reference current, so there is no linear relationship. Therefore, we should decompose the reference current so that a PI controller calculates its peak, which should be constant in the steady state. Now, we have a linear relationship between the input and the output of the PI controller.

4.3.2 Implementation of PI Controller

$$I_{c,peak} = PI(P_{ref} - P) + I_{c,peak,FF}$$

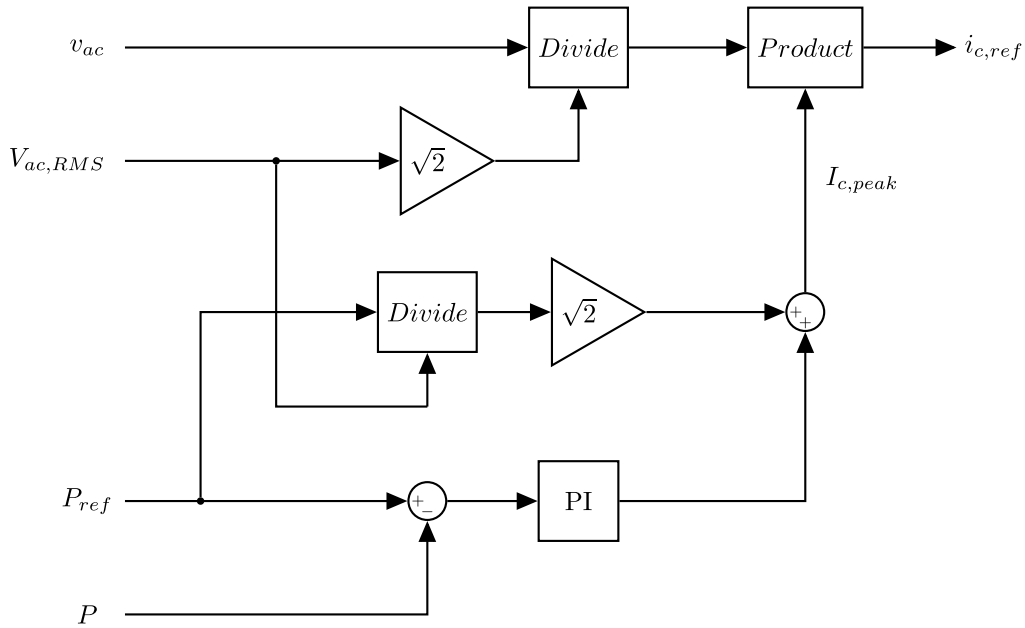
We can also add a feedforward term ($I_{c,peak,FF}$) to improve the response. In an ideal case, we get:

$$I_{c,peak} = PI(P_{ref} - P) + \sqrt{2}\frac{P_{ref}}{V_{ac,RMS}}$$

Units check: [W]/[V] = [A], consistent with a peak current. The $\sqrt{2}$ converts RMS voltage to peak. And then we oscillate the reference current peak to make it sinusoidal.

$$i_{c,ref} = I_{c,peak} \frac{v_{ac}}{\sqrt{2} V_{ac,RMS}}$$

The Block diagram of the power control loop is shown in the figure below.



4.3.3 Numerical Example

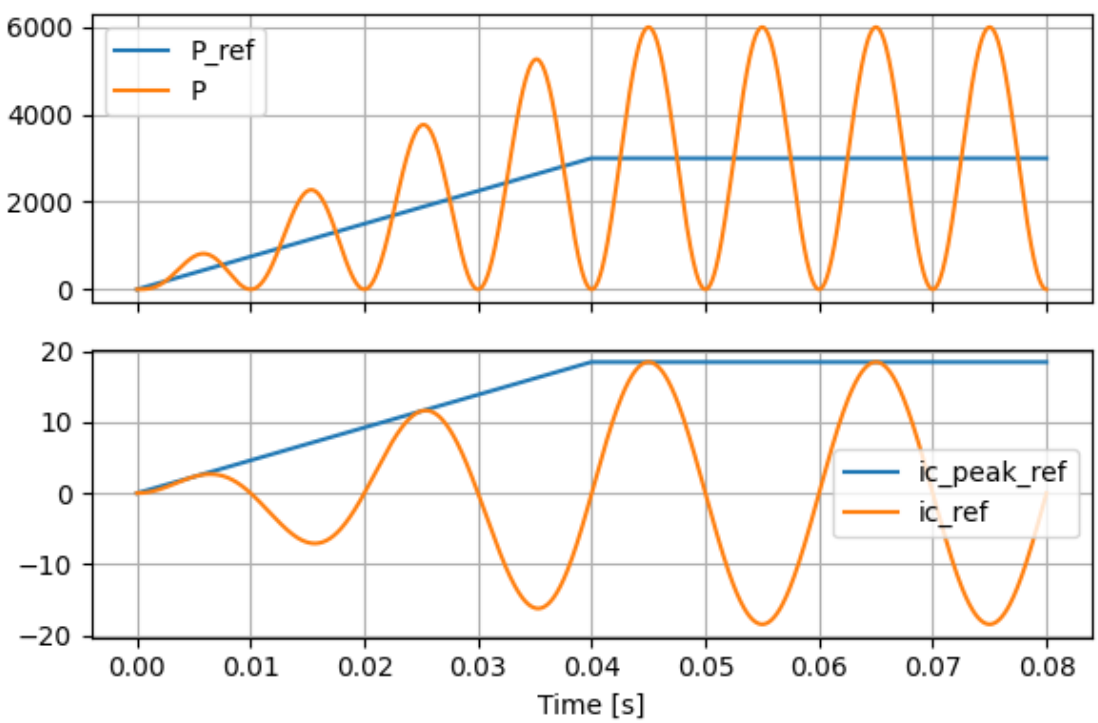
*(This example reuses the imports and grid parameters from the **Common setup** section at the top. Run those cells first.)*

For this ideal model, the feedforward is close to ideal, so you won't see the effect of the PI controller. Therefore, I have made two plots: one with only a feedforward and one with only a PI controller.

Only Feedforward

[10]:
```
# Defining time
t_end_long = 4/fac
N_long = 1000 # Number of samples
t_long = np.linspace(0, t_end_long, N_long)

# Making a sine wave for vac
vac= np.sqrt(2) * Vac * np.sin(2 * np.pi * fac * t_long)

P_ref = 3000 * np.ones(N_long)
P_ref[0:int(N_long/2)] = P_ref[0:int(N_long/2)] * np.linspace(0, 1, int(N_long/
↪2))# making a ramp
```

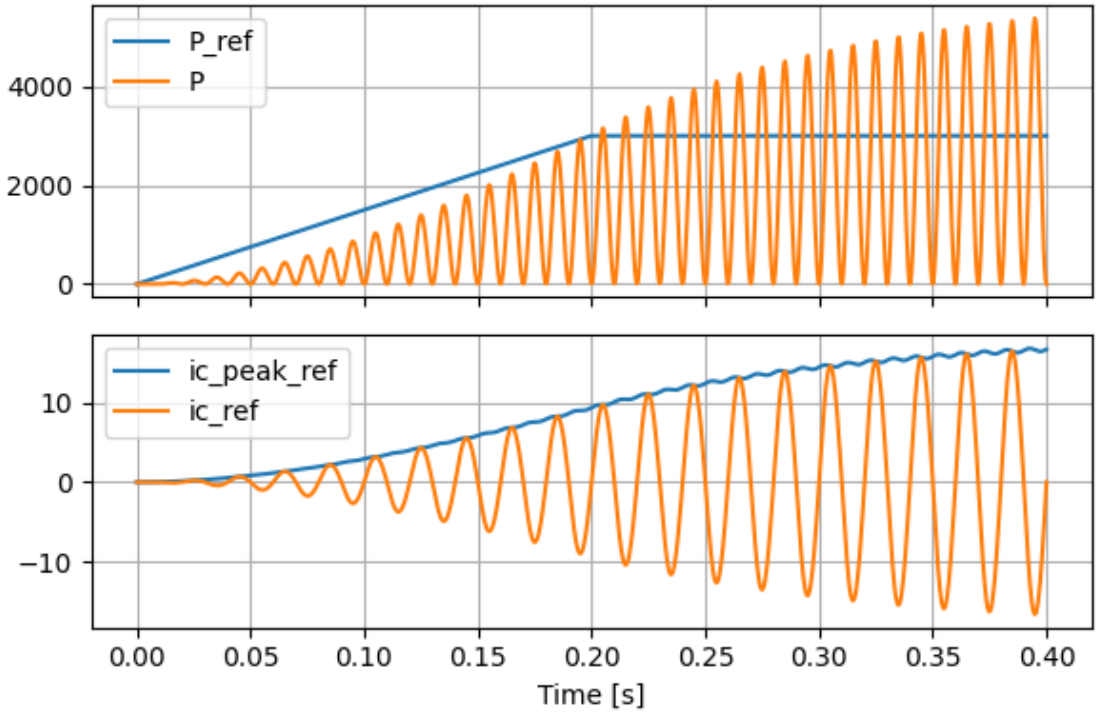
[11]:
```
P = np.zeros(N_long)
ic_peak_ref = np.zeros(N_long)
ic_ref = np.zeros(N_long)
err_cum = 0


ic_peak_ref = np.sqrt(2) * P_ref / Vac

ic_ref = ic_peak_ref * vac / (np.sqrt(2) * Vac)

P = ic_ref * vac


inv_func.plot_time_signals(
    t_long,
    [
        [[P_ref, "P_ref"], [P, "P"]],
        [[ic_peak_ref, "ic_peak_ref"], [ic_ref, "ic_ref"]],
    ],
    height_per_plot=2
)
```



Pure PI controller

[12]:
```
t_end_long = 20/fac
N_long = 1000 # Number of samples
t_long = np.linspace(0, t_end_long, N_long)

# Making a sine wave for vac
vac= np.sqrt(2) * Vac * np.sin(2 * np.pi * fac * t_long)

P_ref = 3000 * np.ones(N_long)
P_ref[0:int(N_long/2)] = P_ref[0:int(N_long/2)] * np.linspace(0, 1, int(N_long/
 ↪2))# making a ramp
```

[13]:
```
kp = 0.00001
ki = 1

P = np.zeros(N_long)
ic_peak_ref = np.zeros(N_long)
ic_ref = np.zeros(N_long)
err_cum = 0

for i in range(1, N_long):
    dt = t[i] - t[i-1]
    err = P_ref[i-1] - P[i-1]
    err_cum += err * dt

    ic_peak_ref[i] = err * kp + err_cum * ki

    ic_ref[i] = ic_peak_ref[i] * vac[i] / (np.sqrt(2) * Vac)

    P[i] = ic_ref[i] * vac[i]


inv_func.plot_time_signals(
    t_long,
    [
        [[P_ref, "P_ref"], [P, "P"]],
        [[ic_peak_ref, "ic_peak_ref"], [ic_ref, "ic_ref"]],
    ],
    height_per_plot=2
)
```



4.3.4 Pros and Cons

✓ **Pros**

- Simple implementation
- No angle estimation (PLL)
- Low computational cost
- Suitable for unity power factor applications (PFC)

✗ **Cons**

- 2ω ripple on power measurement
- Limited flexibility for advanced grid support
- Performance is strongly dependent on grid voltage quality
- No Reactive Power Controller

4.4 DC-Link Voltage Control

The Pure Active Power Control in the previous section assumes you know the desired active power P_{ref}. In a typical PFC rectifier, the primary goal is not a fixed power but a regulated DC-link voltage: the downstream load draws whatever power it needs, and the PFC is expected to draw exactly that much from the grid while keeping v_{dc} constant.

The DC-link voltage controller is an outer loop that sits on top of the Pure Active Power Control. Instead of commanding P_{ref} directly, the user commands $V_{dc,ref}$ and a PI controller generates the current magnitude reference.

4.4.1 Control Method

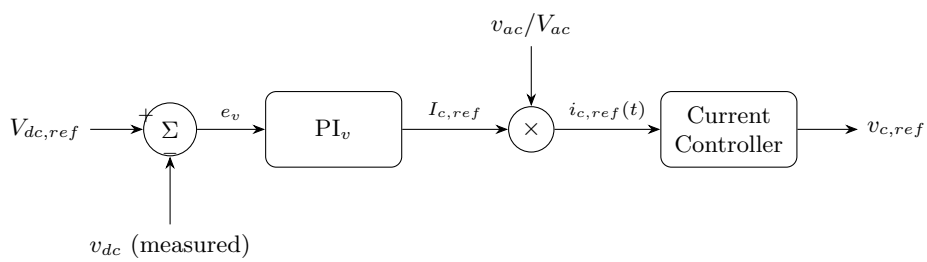
The idea:

1. Measure the DC-link voltage v_{dc}.
2. Compute the error $e_v = V_{dc,ref} - v_{dc}$.
3. A PI controller converts the error into a current magnitude reference $I_{c,ref}$.
4. Shape the current reference using the grid-voltage waveform, as in Pure Active Power Control:

$$i_{c,ref}(t) = \frac{I_{c,ref}}{V_{ac}}\, v_{ac}(t).$$

5. Feed $i_{c,ref}(t)$ into the current controller, which generates the converter voltage reference $v_{c,ref}$ as in Section 4.1.

Because the DC-link capacitor is a slow storage element, the voltage controller runs at a lower bandwidth than the current controller. This bandwidth separation keeps the cascade stable without explicit decoupling.



4.4.2 Pros and Cons

✓ **Pros**

- Automatically matches power flow to the downstream load
- Standard PFC rectifier configuration
- No external power setpoint needed
- Handles load transients through the outer voltage loop

✗ **Cons**

- Adds another control loop (slower outer loop to tune)
- Requires a DC-link model (capacitor, load) for gain selection
- 2ω ripple on the DC-link voltage couples back into $I_{c,ref}$ unless aggressively filtered

5 Comparison of Control Methods

This course presents two outer-loop options that sit above the same current controller: **Pure Active Power Control** (you command P_{ref}) and **DC-Link Voltage Control** (you command $V_{dc,ref}$). Pick the one that matches what your higher-level system already commands.

Outer-loop target:

- Active Power Control: user commands P_{ref} directly
- DC-Link Voltage Control: user commands $V_{dc,ref}$; power is whatever the downstream load draws

Use case:

- Active Power Control: the PFC sits inside a system that already computes a power setpoint (battery charger with an external charge controller, grid services with an external dispatch signal).
- DC-Link Voltage Control: standalone PFC rectifier feeding a downstream DC load or an inverter; the DC bus must be regulated.

Loop structure:

- Active Power Control: single outer loop computing $I_{c,ref}$ from a power setpoint.
- DC-Link Voltage Control: extra outer loop that converts the DC-link error into $I_{c,ref}$; the Active Power Control structure is nested inside.

Tuning effort:

- Active Power Control: one PI (current) plus a feedforward scale factor.
- DC-Link Voltage Control: two PIs (current + voltage) with a bandwidth-separation requirement.

2ω ripple sensitivity:

- Active Power Control: 2ω ripple sits on the power estimate; filter bandwidth trades against response time.
- DC-Link Voltage Control: 2ω ripple sits on v_{dc}; the voltage PI must be slow enough (or the filter aggressive enough) to ignore it, otherwise the ripple propagates into $I_{c,ref}$ and distorts the grid current.